

Travel Itinerary Planner

Written Report for the Multi-Step LLM Agent Assignment

OpenAI LLM calls, Open-Meteo weather tool, hand-rolled Python chain

Problem Statement

A single large prompt cannot do this job well. The model has no access to live weather, so any outdoor plan it invents is a guess about rain and temperature. It also tends to mix two different tasks at once: listing everything a city could offer, and sequencing those ideas across mornings and afternoons. When both are forced into one answer, the list stays shallow and the schedule reads like a brochure. Splitting extraction, forecast retrieval, activity pooling, drafting, critique, and refinement keeps each call focused. Early steps establish facts and constraints; later steps react to structured feedback instead of rewriting from memory. A monolithic prompt also makes debugging difficult: when the output is wrong, you cannot tell whether the dates, the weather, the activity pool, or the pacing failed. This project uses OpenAI for the LLM calls and Open-Meteo for weather, with a shared state object making every dependency visible.

Chain Design

The user text lands in shared state with an injected `today` so relative dates can resolve without the model inventing the current calendar. Step one turns prose into JSON. If the text is nonsense, that step sets `valid` to false and the orchestrator stops before spending tokens on weather or planning. Otherwise step two calls Open-Meteo: geocode the destination, then pull daily highs, lows, precipitation chance, and a short weather-code summary for each trip day that still fits the forecast horizon. Step three expands interests and pace into twelve to twenty candidate activities, still without assigning them to days, so the model cannot hide a weak schedule behind a long bullet list. Step four schedules only from that pool into Markdown days with explicit weather lines. Step five returns JSON critique buckets using the draft, the trip spec, the weather object, and the original activity pool, so it can flag pacing problems as well as invented venues. Step six rewrites the Markdown using the critique, the same pool, and the same weather object, and it ends with a notes section that ties outdoor choices back to specific forecast days.

Each step reads prior keys from state and appends a short trace entry so a demo can narrate the chain from the JSON artifact alone. The separation is strict enough that removing any step leaves a visible hole: no extraction means no coordinates or dates for the tool; no tool means step three guesses climate; no pool means step four invents venues; no critique means first-draft pacing survives; no refine means critique never reaches the traveller-facing document.

Tool Integration

Open-Meteo was chosen because it needs no API key, exposes both geocoding and forecast in simple HTTPS GETs, and returns daily aggregates that map cleanly to itinerary headings. The tool module always returns a dict: success includes `days` with `date`, temperature bounds, precipitation chance, and a human summary derived from WMO codes. Failures keep the same outer shape with a `status` of `unresolved_destination`, `unavailable`, or `out_of_window`, plus a `reason` string. Step three and four prompts branch on that status so the chain still completes when geocoding fails or the trip starts beyond the sixteen-day window. That design trades perfect weather fidelity for a demo that shows graceful degradation instead of a stack trace. When the destination string contains a comma, results are re-ranked so the trailing token can steer country disambiguation, which reduces but does not eliminate wrong-city picks.

Limitations

Forecasts do not extend arbitrarily far ahead, so long-lead trips rely on seasonal language after the horizon. Geocoding can pick the wrong place when a name is ambiguous. The chain does not verify opening hours, tickets, or walking distance between stops; durations in the activity pool are suggestions, not ground truth. Models still drift toward famous sights. The critique pass sometimes lists issues that the refine pass fixes by thinning detail rather than truly rebalancing the day. Those limits are visible in the outputs on purpose; hiding them would pretend the

system is authoritative. Costs and crowd levels are not queried from anywhere, so “mid budget” is interpreted only through prompt wording. The user cannot edit intermediate structures between steps, so a wrong `destination` string in step one propagates unless they rerun the whole chain.

Reflection

Given more time, a second tool for hours or transit would ground claims that are now purely narrative. A self-consistency pass on the activity pool could reduce duplicate ideas. A human confirmation step after extraction would catch mis-geocoded cities before weather runs. During development, a language model helped draft boilerplate and first-pass prompts; it initially proposed a very loose extractor prompt that mixed prose with JSON, which broke the parser and produced unstable field names. That mistake was corrected by tightening the extract prompt and documenting the change in `PROMPTS.md` instead of adding fragile cleanup code in Python. A later audit also found that the critique step was asked to catch invented activities without receiving the activity pool, so that input was added and covered with a regression test. What remains is a small, inspectable pipeline with hand-rolled orchestration, one external tool, and honest reporting of where the weak points still are. The main lesson is that contracts between steps should be enforced where the model reads them, not patched where Python parses them, because every silent repair hides a rubric question about prompt design.